✅1. PROCESSES — THE MUST-KNOWS

📌 *Source: Processes notes*

How programs become processes

1. Preprocessor
2. Compile → .o files
3. Link → executable
4. Run → OS loads & starts main()

Important system calls

- `getpid()` → returns process ID.
- `getppid()` → parent PID.
- `system()` → run any shell command.

Exiting

- `exit(status)` → cleans up, flushes buffers.
- `_exit(status)` → **no cleanup** (used right after `fork()`).
- `atexit(func)` → register cleanup handlers.

Process memory layout

High → Low
[Environment & args]
[Stacks]
[Heap]
[Uninitialized static]
[Initialized static]
[Text / executable]

Process states

New → Ready → Running → Blocked → Done

✅2. SIGNALS — HIGH-YIELD

📌 *Source: Signals notes*

What is a signal?

A *software interrupt* delivered by the OS to notify a process of an event.

Key signals

- **SIGINT** — Ctrl-C
- **SIGALRM** — alarm() fired
- **SIGCHLD** — child terminated
- **SIGKILL** — cannot be caught or ignored
- **SIGPIPE** — pipe read/write failed

- **SIGUSR1 / SIGUSR2** — user-defined

signal()

Registers a handler:

signal(SIGINT, handler);

Special handlers:

- `SIG_IGN` → ignore
- `SIG_DFL` → default behavior

Slow system calls

`read()`, `pause()`, etc.
If interrupted by a signal → return `-1`, `errno = EINTR`.

Important exam points

- **Signals are NOT queued**.
- Handlers must be **simple** & avoid non-reentrant functions.

✅3. PIPES & SELECT

📌 *Source: Pipes notes*

pipe(fd)

`fd[0]` → read end
`fd[1]` → write end

Child inherits pipe FDs after `fork()`.

Properties

- Pipes are **stream-oriented** (not message-oriented).
- If **no writer is open**, read returns 0 (EOF).
- If **no reader is open**, write fails → **SIGPIPE** or errno.

Non-blocking pipe

Use `fcntl(fd, F_SETFL, O_NONBLOCK)`.

If read/write would block → errno = **EAGAIN**.

select()

Monitors multiple FDs for readiness.

Key macros:

- `FD_SET`
- `FD_CLR`
- `FD_ZERO`

- `FD_ISSET`

Timeout:

- `NULL` → wait forever
- `{0,0}` → no wait

✅4. FILE SYSTEM (UNIX SYSTEM CALLS)

📌 *Source: File System Part I & II*

Basic calls

`open`
`read`
`write`
`close`
`lseek`
`unlink` (delete file)

Important rules

- UNIX sees files as **byte streams**.
- System calls return **-1 on error**, errno is set.
- `perror()` used for error reporting.

Sync

- `fsync(fd)` → force write to disk.
- `sync()` → flush all dirty buffers.

Directories

Contain:

- filenames
- inode numbers

✅5. MEMORY MAPPING

📌 *Source: mmap notes*

mmap()

void *mmap(NULL, len, prot, flags, fd, off);

Protections:

- `PROT_READ`
- `PROT_WRITE`

- `PROT_EXEC`
- `PROT_NONE`

Flags:

- `MAP_SHARED` (changes go to file)
- `MAP_PRIVATE` (copy-on-write)
- `MAP_FIXED` (rare)

munmap(addr, len)

Cleans mapping.

Key facts

- Useful for: shared memory, treating file like a struct.
- Cannot extend file size.

✅6. IPC: SHARED MEMORY, SEMAPHORES, MESSAGE QUEUES

📌 *Source: IPC notes*

General structure

IPC uses:

- **key** → user-provided identifier
- **ID** → OS-assigned identifier used for system calls

Shared memory

Fastest form of inter-process communication.

`shmget(key, size, flags)`

Creates or gets an SHM segment.

`shmat(id, NULL, 0)`

Attach to address space (returns pointer).

`shmdt(addr)`

Detach.

`shmctl(id, IPC_RMID)`

Remove — MUST DO to avoid leaks.

✅7. THREADS (POSIX PTHREADS)

📌 *Source: Threads notes*

Why threads?

- Parallel operations
- Simpler design (server example)
- Use multicore CPUs
- Fairness

pthread_create()

pthread_create(&thread, NULL, start_routine, arg);

pthread_exit()

Return from thread.

pthread_join(thread, &retval)

Waits for a thread & retrieves return value.

Thread issues

- Each thread has its own **stack**.
- Must protect shared data.
- Libraries must be **thread-safe**.

✅8. MUTEXES vs SPIN LOCKS (PERFORMANCE)

📌 *Source: mutex/spinlock performance results*

Results summary

- Mutex-protected increments: **~1.7–1.8s**
- No mutex: **~0.09s**
- Spinlock: **13–19 seconds** (!!)

Conclusions

- Mutexes add cost (context switches), **10x slowdown**.
- Threads sharing data may run **slower** due to cache contention.
- Spinlocks are **NOT always faster** — only good if locks are held briefly.

✅9. WHAT YOUR MULTIPLE CHOICE WILL MOST LIKELY TEST

(100% from your notes)

Expect questions on:

- Which signal is uncatchable? (**SIGKILL**)
- What happens when no writer exists for a pipe? (**read → 0 bytes**)
- What system call makes FD non-blocking? (**fcntl**)
- What does `_exit()` NOT do? (**doesn't flush buffers**)
- What is returned when read() interrupted by signal? (**-1, errno = EINTR**)
- mmap protections / flags
- pthread behavior: joining, exiting, arg passing
- Which IPC is fastest? (**shared memory**)
- What select() timeout values mean
- Mutex vs spin lock performance reasoning
- File system's atomicity of read/write
- Difference between MAP_SHARED vs MAP_PRIVATE

✅10. LIKELY PROGRAMMING QUESTIONS

Based on your prof's labs + notes, your coding questions will probably involve one of these:

(A) Pipe parent/child communication

e.g., parent sends string → child reverses → sends back.

You must know:

- create pipe()
- fork()
- close unused ends
- read/write
- maybe use `waitpid()`

(B) Signals with alarms or displaying data

e.g., prime generator with SIGUSR1.

You must know:

- signal()
- alarm()
- volatile variables
- reentrancy rules

(C) Threads + mutex-protected shared variable

Must know:

- pthread_create
- pthread_join
- pthread_mutex_lock/unlock

(D) Shared memory attach → write → detach

Know:

- shmget
- shmat
- shmdt
- shmctl

✅PROCESSES – QUESTIONS & ANSWERS

Q1. What does `fork()` return to the parent and to the child?

A: Parent gets child's PID; child gets 0.

Q2. Why is `_exit()` used after `fork()` instead of `exit()`?

A: `_exit()` does not flush stdio buffers, preventing duplicate flushes from child.

Q3. What does `exec()` do?

A: Replaces the current process image with a new program (does NOT return unless error).

Q4. What happens if you don't `wait()` for a child process?

A: The child becomes a **zombie** until reaped.

Q5. What does `getpid()` return?

A: Current process's PID.

Q6. What does `getppid()` return?

A: Parent process ID.

Q7. What are the 5 main process states?

A: New, Ready, Running, Blocked, Terminated.

Q8. What does the shell do when running your program?

A: Forks → execs program → waits for it.

✅SIGNALS – QUESTIONS & ANSWERS

Q9. What is a signal?

A: A software interrupt delivered to a process by the OS.

Q10. Which signal CANNOT be caught or ignored?

A: SIGKILL.

Q11. What is SIGCHLD used for?

A: Indicates that a **child terminated**.

Q12. What does `signal(SIGINT, handler)` do?

A: Installs a handler for Ctrl-C.

Q13. What happens when a signal interrupts `read()`?

A: `read()` returns -1 and `errno = EINTR`.

Q14. Are signals queued?

A: No. Multiple identical signals may be collapsed into one.

Q15. What must signal handlers avoid?

A: Non-reentrant functions (e.g., `printf`).

Q16. What does `pause()` do?

A: Sleeps until ANY signal is delivered and handled.

Q17. What does `alarm(5)` do?

A: Schedules SIGALRM to occur after 5 seconds.

✅PIPES – QUESTIONS & ANSWERS

Q18. What does `pipe(fd)` create?

A: Two file descriptors:

- `fd[0]` = read
- `fd[1]` = write

Q19. Do pipes require related processes? Why?

A: Yes — child inherits pipe descriptors during `fork()`.

Q20. What does a 0-byte read from a pipe mean?

A: All writers are closed → EOF.

Q21. What happens to `write()` when no readers exist?

A: It fails with SIGPIPE or errno = EPIPE.

Q22. Are pipes message-based or stream-based?

A: Stream-based — you must break messages manually.

Q23. What happens if a pipe is full and you write?

A: The write **blocks** unless FD is non-blocking.

Q24. How do you make a pipe non-blocking?

A: `fcntl(fd, F_SETFL, O_NONBLOCK)`.

✅SELECT() – QUESTIONS & ANSWERS

Q25. What is select() used for?

A: Monitoring multiple FDs for readiness.

Q26. What does a NULL timeout mean?

A: Wait forever.

Q27. What does timeout `{0,0}` mean?

A: Immediate poll — no waiting.

Q28. What does select() return?

A: Number of ready FDs.

Q29. What macros modify FD sets?

A: `FD_SET`, `FD_CLR`, `FD_ZERO`, `FD_ISSET`.

✅FILE SYSTEM – QUESTIONS & ANSWERS

Q30. What does UNIX consider a file?

A: A sequence of bytes (no record structure).

Q31. What system call deletes a file?

A: `unlink()`.

Q32. What does `open()` return?

A: File descriptor (int ≥ 0) or -1 on error.

Q33. What is `errno`?

A: A global variable containing the last error code.

Q34. What does `perror()` do?

A: Prints human-readable error based on errno.

Q35. What is atomic in UNIX?

A: Reads and writes (cannot be half-written).

Q36. What does `fsync(fd)` do?

A: Forces disk write (flush).

Q37. What is a directory file?

A: A list of (filename → inode number) pairs.

✅MEMORY MAPPING – QUESTIONS & ANSWERS

Q38. What does mmap() do?

A: Maps a file into memory.

Q39. What does MAP_SHARED mean?

A: Writes go back to the file and are shared.

Q40. What does MAP_PRIVATE mean?

A: Copy-on-write: changes not written back.

Q41. Why use mmap()?

A: Efficient file I/O and shared memory.

Q42. Can mmap extend file size?

A: No.

Q43. How do you unmap memory?

A: `munmap(addr, len)`.

✅IPC: SHARED MEMORY – QUESTIONS & ANSWERS

Q44. What is the fastest IPC mechanism?

A: Shared memory.

Q45. What does `shmget()` do?

A: Creates or gets a shared memory segment.

Q46. What does `shmat()` return?

A: Pointer to shared memory region.

Q47. Why must you call `shmctl(..., IPC_RMID)`?

A: Shared memory is persistent — otherwise it leaks.

Q48. What is a key in IPC?

A: User-visible identifier used to create/open IPC structures.

✅THREADS – QUESTIONS & ANSWERS

Q49. Why use threads?

A: Parallelism, responsiveness, simplify design, use multiple CPUs.

Q50. What does pthread_create() require?

A: Thread ID pointer, attributes (NULL), start routine, argument.

Q51. What does pthread_join() do?

A: Waits for a thread to finish and retrieves its return value.

Q52. Does each thread get its own stack?

A: Yes.

Q53. What must you protect when using shared variables?

A: Critical sections → use mutexes.

Q54. Why is debugging threaded programs hard?

A: Race conditions vanish/change timing when stepping.

Q55. What makes a function thread-safe?

A: It is reentrant (does not rely on shared state).

✅MUTEXES & SPINLOCKS – QUESTIONS & ANSWERS

Q56. Why are mutexes slow?

A: Context switching & kernel involvement. (10× slowdown seen in tests)

Q57. When are spinlocks better?

A: Only when lock hold time is extremely short.

Q58. Why was spinlock performance worse in tests?

A: Threads wasted CPU cycles spinning → high user time.

Q59. Why can multiple threads accessing the same variable be slower?

A: Cache contention.

✅PROGRAMMING QUESTIONS YOU MUST BE READY FOR

Q60. What steps must you perform to send a message from parent to child?

A:

1. pipe()
2. fork()
3. Parent closes read end, child closes write end
4. write() on parent
5. read() on child

Q61. How do you reverse a string in a child process and send it back?

Use **two pipes**: parent→child and child→parent.

Q62. How do you safely update a shared variable in threads?

Use a pthread mutex:

pthread_mutex_lock(&m);

counter++;

pthread_mutex_unlock(&m);

Q63. How do you write a SIGUSR1 handler that prints a value?

volatile int val;

void handler(int s) { write(1, &val, sizeof(val)); }

Q64. How do you attach to shared memory?

int id = shmget(key, size, 0666);

int *p = (int*)shmat(id, NULL, 0);

✅MORE PROCESS QUESTIONS

Q65. What does `exec()` do to the PID?

A: PID stays the same; only the program image changes.

Q66. What happens to open file descriptors during `exec()`?

A: They remain open unless marked close-on-exec.

Q67. What happens if you call `fork()` and then don't call `_exit()` in the child?

A: Buffered output may be duplicated.

Q68. What happens if the parent dies before the child?

A: Child is adopted by **init** (PID 1).

Q69. What does a return value of -1 from `fork()` mean?

A: The fork failed (usually resource limits).

Q70. Can a zombie process run?

A: No — it's dead but still waiting to have exit status collected.

Q71. What does `waitpid(-1, ...)` mean?

A: Wait for **any** child.

✅MORE SIGNAL QUESTIONS

Q72. What happens if a signal arrives before the handler is installed?

A: The default action applies — it won't "wait" for your handler.

Q73. Why should signal handlers use `volatile` variables?

A: Prevent compiler optimizations that break communication between handler & main code.

Q74. What is the default action for SIGSEGV?

A: Terminate + core dump.

Q75. If a process ignores SIGCHLD, what happens to zombies?

A: They are automatically reaped.

Q76. What does SIGPIPE indicate?

A: Write to pipe/socket with no reader.

Q77. What does `kill(pid, SIGUSR1)` do?

A: Sends SIGUSR1 to that process — does NOT necessarily kill it.

Q78. Can SIGSTOP be caught or ignored?

A: No — like SIGKILL, it's always delivered.

✅MORE PIPE QUESTIONS

Q79. Can pipes be used between unrelated processes?

A: Not directly — must share FD through inheritance or UNIX domain sockets.

Q80. If the parent closes both ends of the pipe, what happens?

A: Child reads EOF instantly.

Q81. How do you send multiple messages over a pipe reliably?

A: Prefix with size or use delimiters (manual framing).

Q82. How big is a Linux pipe usually?

A: 64 KB (based on your notes).

Q83. What happens to write() when pipe buffer fills?

A: Blocks (unless non-blocking).

Q84. When does select() say a pipe is writable?

A: When **at least 1 byte** can be written.

✅MORE SELECT QUESTIONS

Q85. What is nfds in select()?

A: max FD value + 1.

Q86. Does select modify your fd_set?

A: Yes — it marks only ready FDs. Must rebuild sets every call.

Q87. What happens if select() is interrupted by a signal?

A: Returns -1, errno = EINTR.

Q88. Can select monitor files?

A: Yes — but regular files are always considered ready.

Q89. Why isn't select used as a sleep anymore?

A: Better options exist (`nanosleep`, etc).

✅MORE FILE SYSTEM QUESTIONS

Q90. What is the atom of disk access?

A: Blocks.

Q91. What is the atom of UNIX read/write?

A: Bytes.

Q92. What is inode?

A: Metadata describing a file (permissions, size, blocks, etc.)

Q93. Why is unbuffered I/O (system calls) slower?

A: Each call triggers a context switch.

Q94. Why is buffered I/O risky?

A: Data may sit in memory and not be flushed automatically.

Q95. How do you change the working directory?

A: `chdir(path)`.

Q96. What does `getcwd()` return?

A: Absolute path of working directory.

✅MORE MMAP QUESTIONS

Q97. What happens if you write into a MAP_PRIVATE region?

A: Only your process sees it — not written to file.

Q98. Why can mapping be faster than read/write?

A: OS handles paging; avoids user-kernel copying.

Q99. Why should `addr` argument usually be NULL?

A: Let OS choose a proper aligned address.

Q100. What if mmap fails?

A: Returns `MAP_FAILED` (which is `(void*)-1`).

Q101. Can multiple processes map the same file?

A: Yes — allows shared memory via files.

✅MORE IPC: SHARED MEMORY QUESTIONS

Q102. Why is shared memory dangerous?

A: No built-in synchronization — must use semaphores/mutexes.

Q103. What happens if two processes pick the same key accidentally?

A: They attach to the same shared segment → BAD.

Q104. What does `ftok(path, id)` do?

A: Generates a key based on file + project ID.

Q105. What does IPC_PRIVATE (0) mean?

A: Create private shared memory accessible only by parent + children.

Q106. Can a process detach from shared memory without deleting it?

A: Yes — deletion is separate (`shmctl`).

✅MORE THREADING QUESTIONS

Q107. What does pthread_exit(NULL) do?

A: Ends the calling thread without ending the whole program.

Q108. What if main() finishes before threads?

A: Process ends & remaining threads die unless you call pthread_join.

Q109. How do you pass multiple values into a thread?

A: Create a struct; pass its pointer as void* arg.

Q110. Can two threads use the same pointer?

A: Yes — but requires synchronization.

Q111. What causes race conditions?

A: Unsynchronized access to shared data.

Q112. Why is printf dangerous in threads?

A: It uses shared internal state → potential race or corruption.

✅MORE MUTEX/SPINLOCK QUESTIONS

Q113. What does a mutex guarantee?

A: Mutual exclusion — only one thread at a time may enter a critical section.

Q114. What happens if you lock a mutex twice in the same thread?

A: Deadlock (unless it's a recursive mutex).

Q115. Why did spinlocks perform terribly in your professor's test?

A: All threads are continuously spinning → CPU wasted.

Q116. Why does "one thread at a time" outperform multi-threaded increments?

A: Cache locality — only one core modifies the shared variable, no ping-ponging.

Q117. When is a mutex preferred over spinlock?

A: When wait time may be long.

Q118. Why must a child close the write end of a pipe?

A: To ensure parent sees EOF when finished.

Q119. How does parent avoid blocking on read() from pipe?

A: Use non-blocking mode or select().

Q120. Why can't signals be used for large data transfer?

A: They carry no payload.

Q121. Why are directories special files?

A: They store inode mappings, not data content.

Q122. Why is pthread_join required?

A: Ensures thread termination & resource cleanup.

Q123. Can threads call fork()?

A: Yes but dangerous — only the calling thread survives in child.

Q124. Why do mmap'd changes appear instantly to other processes?

A: They reference the same physical memory pages (MAP_SHARED).